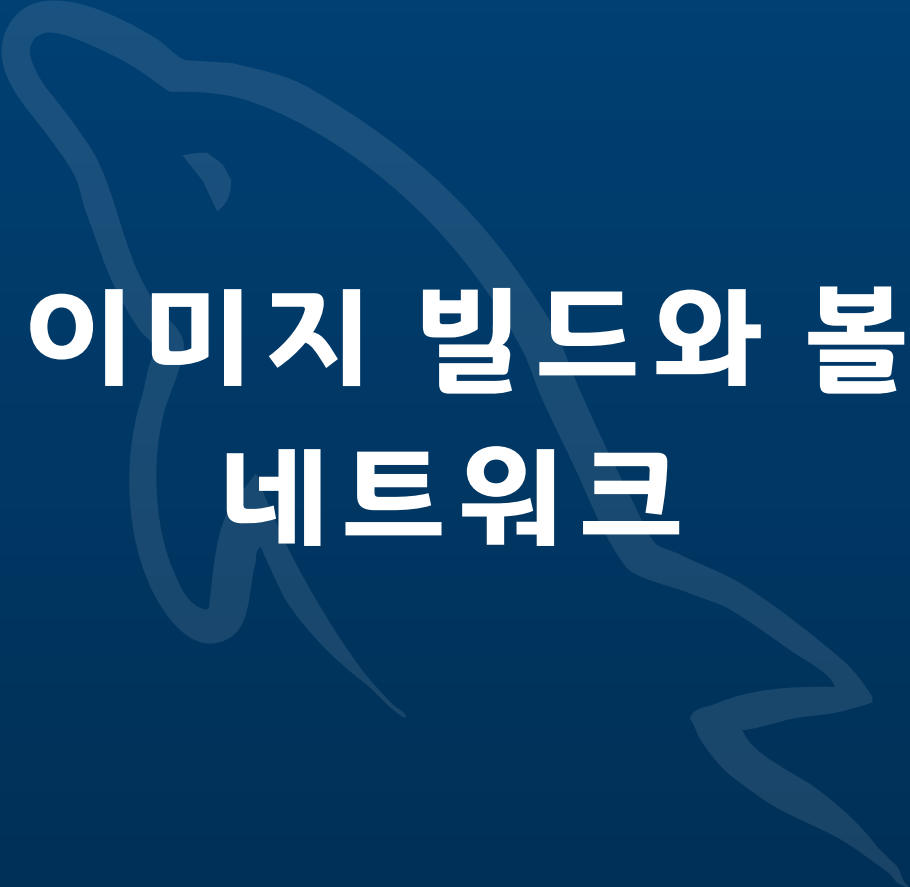




도커 이미지 빌드와 볼륨 · 네트워크

학습 목표

- 호스트에 빌드 도구를 설치하지 않고 멀티 스테이지 빌드로 이미지를 만듭니다
- 하나의 불변 이미지에 설정 파일만 교체해 외부 클라우드 DB와 로컬 DB를 오갑니다
- 명명된 볼륨과 바인드 마운트로 컨테이너 삭제 후에도 데이터를 보존합니다
- 사용자 정의 네트워크에서 컨테이너 이름으로 DB에 접속합니다

애플리케이션 패키징의 핵심은 호스트 환경과 무관한 **멀티 스테이지 빌드**, 코드와 설정을 분리하는 **12-Factor 환경변수 주입**, 그리고 볼륨과 사설 네트워크를 통한 **상태 영속화와 격리**입니다. 본 차시는 이 세 가지를 하나의 스프링부트 애플리케이션에 차례로 적용합니다.

핵심 개념 및 구조

도커 이미지의 레이어 구조

도커 이미지는 하나의 통짜 파일이 아니라, `Dockerfile` 지시어마다 만들어진 읽기 전용 레이어가 순서대로 쌓인 구조입니다.

Dockerfile 지시어	생성되는 레이어	캐시 재사용 조건
<code>FROM azul/zulu-openjdk-alpine</code>	베이스 런타임	태그가 같으면 재사용
<code>WORKDIR /app</code>	작업 경로 메타데이터	경로가 같으면 재사용
<code>COPY build/libs/*.jar app.jar</code>	애플리케이션 산출물	복사 대상 파일이 바뀌면 무효화
<code>ENTRYPOINT ["java", "-jar", "app.jar"]</code>	실행 명령 메타데이터	명령이 같으면 재사용

레이어는 모두 읽기 전용이며, 컨테이너를 실행하면 그 위에 쓰기 가능한 레이어가 한 장 더 얹힙니다. `docker rm`으로 삭제되는 것은 이 쓰기 레이어이며, 여기에 쌓인 데이터도 함께 사라집니다.

빌드 캐시와 COPY 순서

레이어 하나가 무효화되면 그 아래 모든 레이어가 다시 실행됩니다. 자주 바뀌는 파일을 늦게 복사할수록 캐시가 오래 유지됩니다.

소스와 함께 복사한 경우

```
COPY . .  
RUN gradle bootJar
```

소스 한 줄만 고쳐도 의존성 다운로드부터 전부 다시 수행합니다.

빌드 스크립트를 먼저 복사한 경우

```
COPY build.gradle settings.gradle .  
RUN gradle dependencies --no-daemon  
COPY src src  
RUN gradle bootJar --no-daemon
```

의존성이 그대로면 앞 레이어를 재사용하고 컴파일만 다시 합니다.

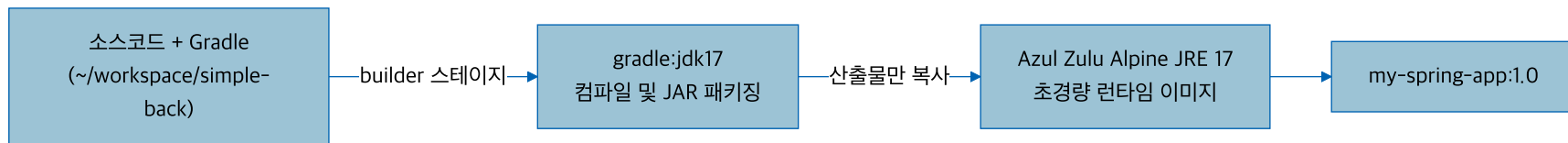
캐시는 **지시어 문자열과 복사 대상 파일의 내용**이 모두 같을 때만 재사용됩니다. 한 레이어가 무효화되면 그 아래는 조건과 무관하게 전부 다시 실행되므로, 변경 빈도가 낮은 파일부터 복사하는 것이 [Dockerfile](#) 작성의 기본 순서입니다.

단일 스테이지와 멀티 스테이지 빌드 비교

비교 항목	단일 스테이지 (호스트 아티팩트 복사)	멀티 스테이지 빌드 (실무 표준)
호스트 요구사항	특정 버전 JDK와 빌드 도구 사전 설치	도커 엔진만 있으면 완결
환경 일관성	개발자 PC 차이로 빌드 결과 파편화	표준 빌더 컨테이너에서 동일 결과
빌드 절차	<code>./gradlew bootJar</code> 후 이미지 빌드 2단계	<code>docker build</code> 한 번으로 완결
최종 이미지	빌드 도구와 캐시가 남을 수 있음	산출물과 JRE만 남아 150~180MB

단일 스테이지는 빌드 환경을 호스트에 의존하므로 학습자마다 결과가 달라질 수 있습니다. 멀티 스테이지는 컴파일 환경까지 이미지 안으로 들여와 호스트 종속성을 제거합니다.

멀티 스테이지 빌드 파이프라인



✂ 1단계 빌더

JDK 17과 Gradle이 갖춰진 공식 이미지에서 컴파일을 수행

⬆ 산출물 전달

`COPY --from=builder`로 JAR 한 개만 다음 단계로 전달

▶ 2단계 런타임

컴파일러가 빠진 Alpine JRE 위에서 실행하며 `amd64` · `arm64`를 모두 지원

런타임 베이스 이미지 선택

같은 Alpine 계열이라도 OpenJDK 배포판마다 제공하는 CPU 아키텍처가 다릅니다.

⚠ Eclipse Temurin

eclipse-temurin:17-jre-alpine

JDK 17 Alpine 태그는 `linux/amd64`만 배포합니다. Alpine 멀티 아키텍처는 JDK 21부터 제공됩니다.

✓ Azul Zulu

azul/zulu-openjdk-alpine:17-jre-headless

JDK 17에서도 `linux/amd64`와 `linux/arm64`를 함께 배포해 두 아키텍처에서 그대로 빌드됩니다.

Temurin Alpine 이미지를 Apple Silicon에서 빌드하면 `no match for platform in manifest: not found`로 중단됩니다. `--platform linux/amd64`를 붙이면 에뮬레이션으로 우회할 수 있지만, 실습에서는 두 아키텍처를 모두 지원하는 Azul Zulu Alpine을 표준으로 사용합니다.

이미지 태그 전략

하나의 빌드 산출물에 `-t`를 중복 지정하면 서로 다른 태그를 동시에 부여할 수 있습니다.

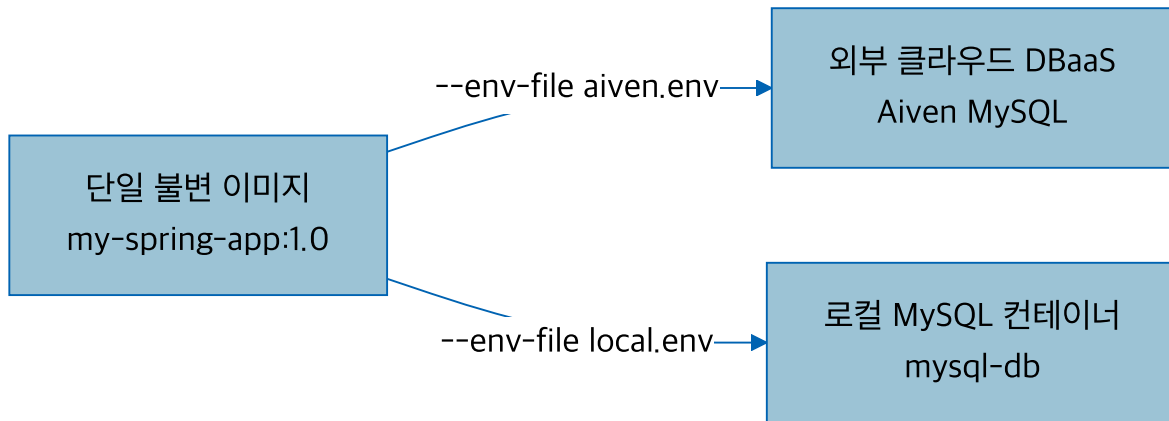
```
docker build -t my-spring-app:1.0 -t my-spring-app:prod .
```

태그	성격	실무 용도
<code>1.0</code>	불변 릴리스 식별자	배포 추적과 롤백의 기준
<code>prod</code> / <code>dev</code>	환경 별칭	환경별로 가리키는 대상을 전환
<code>latest</code>	가변 포인터	편의용 별칭. 운영 배포 기준으로는 부적합

`latest`는 언제든지 다른 이미지로 덮어써질 수 있어 어떤 변경이 배포되었는지 추적할 수 없습니다. 운영 배포는 시맨틱 버전이나 커밋 해시 같은 불변 태그를 기준으로 합니다.

코드와 설정의 분리 (12-Factor App)

접속 정보를 이미지에 고정하면 환경마다 이미지를 새로 빌드해야 합니다. 설정을 밖으로 빼면 같은 이미지가 그대로 여러 환경에 붙습니다.



이미지는 다시 빌드하지 않고 주입하는 설정 파일만 교체합니다. 빌드 산출물이 환경마다 달라지지 않으므로, 로컬에서 검증한 이미지를 그대로 운영에 올릴 수 있습니다.

환경변수 주입 방식 비교

주입 방식	값이 적용되는 시점	특징과 한계
<code>ENV</code> (Dockerfile)	이미지 빌드 시점에 고정	비민감 기본값만 허용. 자격증명 하드코딩 금지
<code>-e KEY=VALUE</code>	컨테이너 실행 시점	변수가 늘면 명령이 길어지고 암호가 쉘 히스토리에 남음
<code>--env-file 파일</code>	컨테이너 실행 시점	실무 표준. 파일 교체만으로 환경 전환

데이터베이스 URL과 비밀번호는 `Dockerfile`에 `ENV`로 넣지 않습니다. 이미지는 외부에 공유되는 읽기 전용 산출물이기 때문입니다. 환경변수 파일은 `.gitignore`에 등록하고 저장소에는 `.env.example` 템플릿만 공유합니다.

컨테이너 데이터 영속성

저장 방식	데이터 실제 위치	컨테이너 삭제 시	주 용도
볼륨 없음	컨테이너 쓰기 레이어	함께 영구 삭제	상태를 갖지 않는 애플리케이션
명명된 볼륨	도커 엔진 관리 영역	보존	데이터베이스 영속 스토리지 표준
바인드 마운트	호스트의 지정한 절대경로	보존	파일 직접 확인, 설정 파일 주입

볼륨 없이 실행한 MySQL 컨테이너를 `docker rm`으로 삭제하면 그동안 쌓인 테이블과 레코드가 모두 영구 소멸합니다. 상태를 갖는 데이터베이스에는 반드시 볼륨을 지정합니다.

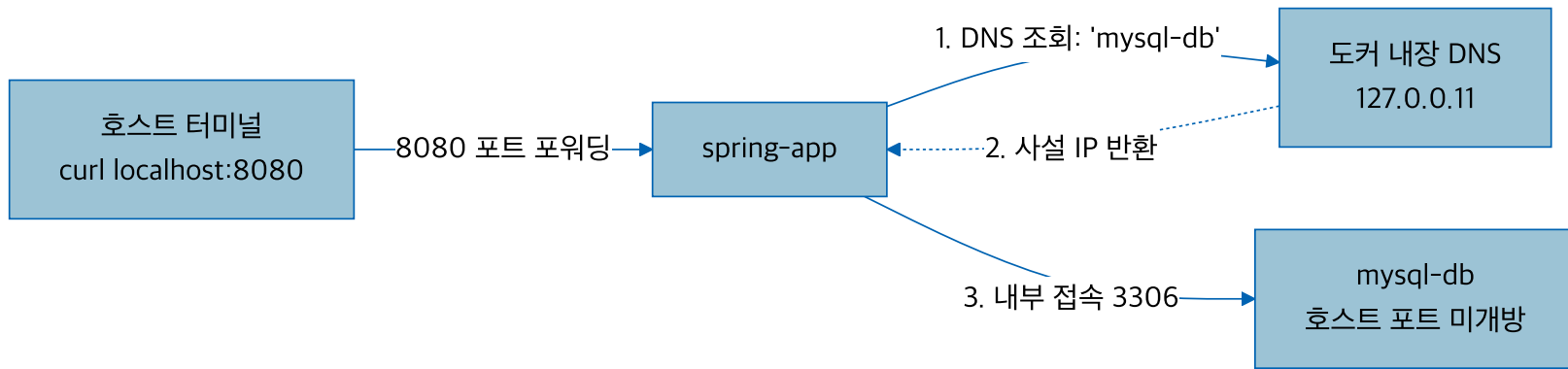
컨테이너 네트워크 핵심 개념

호스트와 격리된 컨테이너가 외부와 통신하고, 컨테이너끼리 안전하게 대화하기 위한 필수 개념입니다.

네트워크 개념	현실 개념 (매핑)	기술 정의 및 실습 역할
호스트 (Host)	컴퓨터 본체 / 건물 전체	도커 엔진이 구동되는 실제 머신 OS (호스트 터미널, 브라우저)
포트 (Port)	건물의 출입구 문번호	실행 중인 프로세스를 식별하는 통신 번호 (<code>8080</code> , <code>3306</code>)
포트 포워딩 (<code>-p</code>)	건물 로비와 개별 방 연결	호스트 포트 요청을 컨테이너 내부 포트로 전달 (<code>localhost:8080</code>)
사설 네트워크	단지 내부 전용 통신망	호스트 외부에 노출되지 않고 내부에서만 통하는 사설망 (<code>app-net</code>)
사용자 정의 브리지	전용 아파트 단지	컨테이너 간 사설 격리와 내장 DNS 조회(<code>127.0.0.11</code>)를 제공하는 가상망

사용자 정의 네트워크와 DNS 서비스 디스커버리

도커 내장 DNS(전화번호부)가 컨테이너 이름을 사실 IP로 자동 해석합니다. 컨테이너가 재시작되어 IP가 바뀌어도 애플리케이션 연결이 유지됩니다.



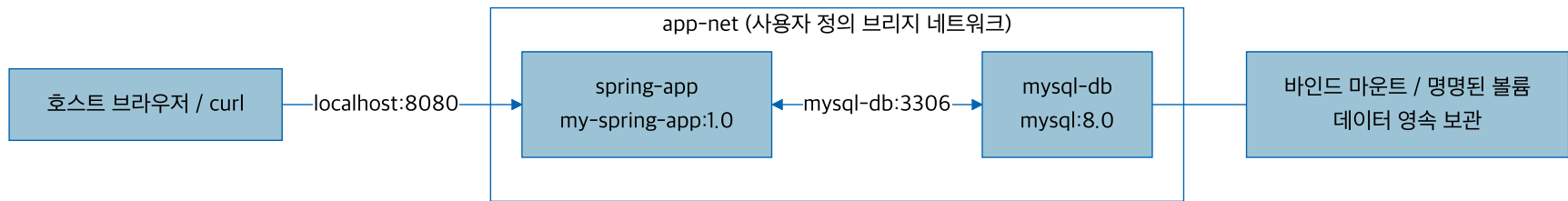
기본 bridge vs 사용자 정의 네트워크

기본 `bridge` 는 내장 DNS가 없어 IP로만 통신해야 합니다. 사용자 정의 네트워크 (`docker network create`)에서만 이름 기반 조회가 동작합니다.

동적 IP 추적과 포트 격리

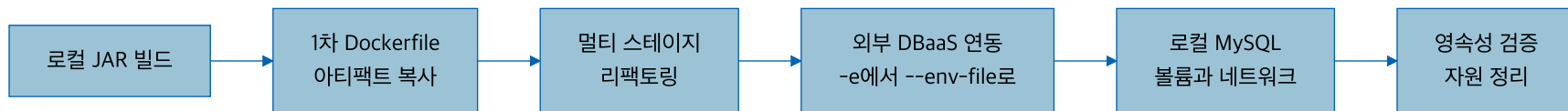
컨테이너 재생성 시 사실 IP가 변경되어도 이름으로 자동 추적합니다. DB 포트(`3306`)를 호스트 외부에 열지 않아도 안전하게 격리 연동됩니다.

컨테이너 런타임 구조 종합



애플리케이션은 **포트 포워딩**으로 외부에 열리고, 데이터베이스는 **사설 네트워크 안에서만** 접근되며, 데이터는 **컨테이너 바깥**에 납습니다.

실습 전체 흐름: 로컬 빌드부터 연속성 검증까지



≡ 이미지 빌드

호스트 의존 빌드에서 멀티 스테이지 빌드로 전환하고 태그를 부여

☞ 설정 주입

같은 이미지에 `--env-file` 만 바꿔 접속 대상을 교체

📖 상태 관리

사설 네트워크로 격리하고 볼륨으로 데이터를 보존

실습 단계

실습 로드맵: 이미지 빌드와 설정 주입

Step	주제	이 단계에서 직접 확인하는 것
1	로컬 JAR 아티팩트 빌드	Gradle 래퍼로 실행 가능한 JAR 생성과 산출물 위치
2	아티팩트 기반 1차 Dockerfile	호스트 빌드 결과를 복사하는 단일 스테이지 구조
3	멀티 스테이지 리팩터링	호스트에 JDK 없이 <code>docker build</code> 한 번으로 완결
4	외부 DBaaS 연동 (<code>-e</code>)	명령줄 환경변수 주입과 커넥션 풀 연결 로그
5	<code>--env-file</code> 전환	설정 파일 분리로 자격증명 노출 위험 제거

Step 1~3은 **같은 애플리케이션을 세 가지 방식으로 이미지화**하며 차이를 비교하고, Step 4~5는 이미지를 다시 빌드하지 않고 설정만 교체합니다.

실습 로드맵: 네트워크·볼륨과 영속성 검증

Step	주제	이 단계에서 직접 확인하는 것
6	사용자 정의 네트워크와 명명된 볼륨	도커 관리 스토리지 생성과 <code>Mountpoint</code> 조회
7	바인드 마운트와 백그라운드 DB 제어	호스트 디렉터리에 생성되는 실제 DB 파일
8	로컬 MySQL 연동 설정 주입	동일 이미지에 <code>local.env</code> 만 교체해 대상 전환
9	DNS 해석과 API 엔드포인트 검증	컨테이너 이름 기반 접속과 HTTP 응답 확인
10	삭제 후 영속성 검증과 자원 정리	컨테이너 삭제 후에도 남는 데이터 파일

Step 6~7은 데이터 저장 위치를 **도커 관리 영역**과 **호스트 디렉터리** 두 방식으로 나누어 구성하고, Step 10에서 그 차이를 삭제 실험으로 검증합니다.

명령어 한눈에 보기: 로컬 빌드와 1차 이미지

로컬 아티팩트 빌드 Step 1

```
# 프로젝트 루트로 이동
cd ~/workspace/simple-back
ls -la

# Gradle 래퍼로 실행 가능 JAR 생성
./gradlew bootJar

# 산출물 파일명과 용량 확인
ls -lh build/libs/
```

아티팩트 복사 이미지 빌드 Step 2

```
# 단일 스테이지 Dockerfile 작성
FROM azul/zulu-openjdk-alpine:17-jre-headless
WORKDIR /app
COPY build/libs/*-SNAPSHOT.jar app.jar
ENV PORT=8080
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]

# 버전·환경 태그 동시 부여 빌드
docker build -t my-spring-app:0.1 \
  -t my-spring-app:dev .
docker images my-spring-app
```

멀티 스테이지 Dockerfile 최종본

1단계: 빌드 환경 `builder`

```
FROM gradle:jdk17 AS builder
WORKDIR /workspace

# 의존성 캐시 최적화:
# 소스보다 빌드 스크립트를 먼저 복사
COPY build.gradle settings.gradle .
RUN gradle dependencies --no-daemon || true

# 소스 복사 후 JAR 패키징
COPY src src
RUN gradle bootJar --no-daemon
```

2단계: 실행 환경 `runtime`

```
FROM azul/zulu-openjdk-alpine:17-jre-headless
WORKDIR /app

# 빌드 산출물만 복사
# 소스코드와 빌드 도구는 제외된다
COPY --from=builder \
    /workspace/build/libs/*-SNAPSHOT.jar app.jar

# 비민감 기본값만 선언
ENV PORT=8080

EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

DB 접속 URL과 비밀번호 같은 민감한 설정은 `ENV`로 이미지에 넣지 않습니다. 이미지는 외부에 공유되는 읽기 전용 산출물이므로 `PORT` 같은 비민감 기본값만 선언합니다.

명령어 한눈에 보기: 멀티 스테이지 빌드와 태그

호스트 종속성 배제 검증 Step 3

```
# 로컬 빌드 산출물을 먼저 제거한다
./gradlew clean
rm -rf build/

# 호스트에 JDK가 없어도 완결되는 빌드
docker build \
  -t my-spring-app:1.0 \
  -t my-spring-app:prod .
```

태그와 이미지 크기 확인

```
# 동일 이미지 ID에 두 태그가 매핑된다
docker images my-spring-app

# 레이어 구성과 용량 확인
docker history my-spring-app:1.0

# 빌드 단계까지만 실행 (선택)
docker build --target builder \
  -t my-spring-app:builder-cache .
```

빌드 컨텍스트 인자인 마지막 마침표는 **COPY**의 기준 경로입니다. 태그는 릴리스 버전과 배포 환경을 함께 부여해 하나의 빌드로 두 식별자를 만듭니다.

명령어 한눈에 보기: 환경변수 주입 전환

명령줄 직접 주입 Step 4

```
docker run -d --name spring-cloud-db \
  -p 8080:8080 \
  -e SPRING_DATASOURCE_URL="$AIVEN_URL" \
  -e SPRING_DATASOURCE_USERNAME="$AIVEN_USER" \
  -e SPRING_DATASOURCE_PASSWORD="$AIVEN_PASSWORD" \
  my-spring-app:1.0
```

```
docker ps
docker logs spring-cloud-db | grep "HikariPool"
curl -i http://localhost:8080/users
```

설정 파일 일괄 주입 Step 5

```
# aiven.env (자격증명은 파일로 분리)
PORT=8080
SPRING_DATASOURCE_URL=jdbc:mysql://<호스트>:<포트>/defaultdb
SPRING_DATASOURCE_USERNAME=avnadmin
SPRING_DATASOURCE_PASSWORD=발급받은_비밀번호
```

```
# 파일 한 줄로 일괄 주입
docker run -d --name spring-cloud-db \
  -p 8080:8080 \
  --env-file aiven.env \
  my-spring-app:1.0
```

`-e`로 전달한 비밀번호는 쉘 히스토리와 프로세스 목록에 남습니다. 실제 환경변수 파일은 `.gitignore`에 등록하고 템플릿인 `.env.example`만 저장소에 공유합니다.

명령어 한눈에 보기: 네트워크·볼륨과 로컬 DB 연동

네트워크·볼륨 구성 Step 6~7

```
# 사실 브리지 네트워크 생성
docker network create app-net

# 명명된 볼륨 (도커 관리 영역)
docker volume create mysql-named-data
docker volume inspect mysql-named-data

# 바인드 마운트 (호스트 디렉터리)
mkdir -p ~/workspace/simple-back/mysql-host-data
docker run -d --name mysql-db --network app-net \
  -e MYSQL_ROOT_PASSWORD=설정할_비밀번호 \
  -e MYSQL_DATABASE=mydb \
  -v ~/workspace/simple-back/mysql-host-data:/var/lib/mysql \
  mysql:8.0
```

로컬 MySQL 연동 Step 8

```
# local.env (호스트명이 컨테이너 이름)
PORT=8080
SPRING_DATASOURCE_URL=jdbc:mysql://mysql-db:3306/mydb
SPRING_DATASOURCE_USERNAME=root
SPRING_DATASOURCE_PASSWORD=발급받은_비밀번호

# 같은 네트워크에 애플리케이션 배치
docker run -d --name spring-app \
  --network app-net \
  -p 8080:8080 \
  --env-file local.env \
  my-spring-app:1.0

docker ps
```

연동 검증과 자주 만나는 오류

증상	확인 명령	주요 원인
<code>UnknownHostException</code>	<code>docker network inspect app-net</code>	두 컨테이너가 다른 네트워크에 있음
<code>Access denied for user</code>	<code>docker logs spring-app</code>	계정·비밀번호 불일치
<code>Connection refused</code>	<code>docker logs -f mysql-db</code>	DB 초기화 미완료 상태에서 접속
<code>Communications link failure</code>	<code>docker ps</code>	DB 컨테이너가 종료된 상태

정상 연동 시 `docker logs spring-app`에 `HikariPool-1 - Start completed.`가 출력됩니다.

데이터 영속성 검증과 자원 정리

삭제 후 데이터 확인 Step 10

```
# 컨테이너 정지 및 영구 삭제
docker stop spring-app mysql-db
docker rm spring-app mysql-db
docker ps -a

# 호스트 디렉터리의 DB 파일은 그대로 유지
ls -la ~/workspace/simple-back/mysql-host-data/

# 명명된 볼륨도 독립적으로 남는다
docker volume ls
```

실습 자원 정리

```
# 사설 네트워크 제거
docker network rm app-net

# 명명된 볼륨 제거 (데이터 영구 삭제)
docker volume rm mysql-named-data

# 남은 자원 최종 확인
docker network ls
docker volume ls
docker images my-spring-app
```

컨테이너 삭제는 데이터 삭제가 아닙니다. 반대로 `docker volume rm`은 데이터를 영구 삭제하므로, 운영 환경에서는 볼륨 제거 전 백업 여부를 반드시 확인합니다.

Step별 이해 포인트: 빌드와 설정 주입

Step	반드시 이해하고 넘어갈 핵심 포인트
1	JAR는 호스트 JDK로 만든 산출물이므로 호스트 환경에 종속됩니다.
2	<code>COPY</code> 는 빌드 컨텍스트 기준 경로에서만 파일을 가져올 수 있습니다.
3	빌드 스크립트를 소스보다 먼저 복사해야 의존성 캐시가 재사용됩니다.
4	<code>-e</code> 는 간편하지만 비밀번호가 쉘 히스토리에 그대로 남습니다.
5	이미지는 그대로 두고 설정 파일만 교체해 접속 대상을 바꿉니다.

Step 3에서 `rm -rf build/`로 로컬 산출물을 지운 뒤에도 빌드가 성공한다는 점이 **호스트 독립성**의 직접적인 증거입니다.

Step별 이해 포인트: 네트워크와 영속성

Step	반드시 이해하고 넘어갈 핵심 포인트
6	명명된 볼륨은 도커 관리 영역에 저장되어 호스트 경로에 의존하지 않습니다.
7	바인드 마운트는 호스트 디렉터리를 그대로 쓰므로 파일을 직접 열람합니다.
8	JDBC URL의 호스트명이 IP가 아니라 컨테이너 이름으로 선언됩니다.
9	같은 네트워크 안이면 DB 포트를 외부로 노출하지 않아도 연결됩니다.
10	컨테이너를 지워도 볼륨과 호스트 디렉터리의 데이터는 남습니다.

볼륨 없이 실행한 DB 컨테이너를 삭제하면 그동안 쌓인 테이블과 레코드가 함께 영구 소멸합니다. 상태를 가지는 서비스에는 반드시 볼륨을 지정합니다.

한 장 요약

영역	핵심 요약	비고
빌드 방식	빌더 스테이지에서 컴파일하고 산출물만 런타임으로 복사	호스트에 JDK 불필요, 최종 이미지 경량화
태그 전략	릴리스 버전과 배포 환경 태그를 단일 빌드로 동시 부여	<code>-t</code> 중복 지정, <code>:latest</code> 단독 사용 지양
설정 주입	코드와 설정을 분리해 동일 이미지로 환경 전환	<code>-e</code> 에서 <code>--env-file</code> 로 전환
상태 관리	명명된 볼륨과 바인드 마운트로 데이터 영속화	컨테이너 수명과 데이터 수명의 분리

하나의 불변 이미지에 설정 파일만 교체해 외부 DBaaS와 로컬 MySQL을 오가는 구성이 12-Factor App 설정 분리 원칙의 실제 적용 형태입니다.

Appendix

선택 빌드 분기

```
# 멀티 스테이지의 특정 단계까지만 빌드
docker build --target builder -t my-spring-app:builder-cache .

# 별도 Dockerfile을 지정해 빌드
docker build -f Dockerfile.dev -t my-spring-app:dev .
```

분기 방식	동작	선택 기준
<code>--target [스테이지]</code>	지정한 스테이지까지만 실행하고 중단	베이스 설정을 공유하며 단계만 잘라 쓸 때
<code>-f [파일명]</code>	다른 Dockerfile을 빌드 정의로 사용	환경별 빌드 절차 자체가 크게 다를 때

파일을 분리하면 직관적이지만 공통 설정이 중복됩니다. 단일 Dockerfile에 스테이지를 정의하고 `--target`으로 분기하는 방식이 유지보수에 유리합니다.

심화 네트워크: 포트 포워딩 원리와 드라이버

포트 포워딩 저수준 원리 (iptables NAT)

`-p 8080:8080` 지정 시 도커 데몬이 Linux 커널 `iptables`의 `PREROUTING` 체인에 DNAT 규칙을 등록합니다. 호스트 포트로 유입된 패킷 목적지를 컨테이너 사설 IP로 변환해 가상 브리지 인터페이스로 전달합니다.

도커 네트워크 드라이버 심화 비교

- `bridge`: 가상 사설망 격리 및 NAT 패킷 중계 (실무 표준)
- `host`: 호스트 네트워크 네임스페이스 직접 공유
- `none`: 로컬 루프백 외 외부 통신 완전 차단
- `overlay`: 분산 다중 호스트 간 가상 네트워크 평면 구축
- `macvlan`: 물리 MAC 주소를 부여해 독립 장비 취급

핵심 기술 면접 질문

호스트 아티팩트 복사 vs 멀티 스테이지 빌드

스프링부트 애플리케이션을 도커라이징할 때 호스트에서 빌드된 JAR를 복사하는 대신 멀티 스테이지 빌드를 적용해야 하는 결정적인 이유는 무엇인가요?

환경변수 주입의 아키텍처 이점

애플리케이션 컨테이너에서 데이터베이스 접속 정보를 소스코드나 Dockerfile에 하드코딩하지 않고 환경변수로 주입하는 이유는 무엇인가요?

명명된 볼륨 vs 바인드 마운트

도커에서 경로를 지정하지 않는 볼륨(Named Volume)과 호스트 경로를 직접 지정하는 볼륨(Bind Mount)의 구조적 차이와 실무 활용 기준은 무엇인가요?

DB 컨테이너 볼륨 미지정 시의 위험

DB 컨테이너에 볼륨을 마운트하지 않고 실행했을 때 발생하는 치명적인 위험은 무엇인가요?

핵심 기술 면접 질문

사용자 정의 네트워크의 보안 이점

스프링부트와 데이터베이스 컨테이너를 연동할 때 호스트 포트 포워딩(`-p 3306:3306`) 없이 사용자 정의 네트워크를 사용하는 이유는 무엇인가요?

기본 브리지 vs 사용자 정의 네트워크의 DNS 해석

도커 기본 네트워크(default bridge)와 사용자 정의 네트워크(user-defined network)의 DNS 이름 해석(Service Discovery) 동작 차이는 무엇인가요?

iptables NAT 기반 포트 포워딩 원리

도커 포트 포워딩(`-p`)이 호스트 OS 수준에서 실제로 패킷을 전달하는 저수준 iptables NAT 원리는 무엇인가요?

`-e` vs `--env-file` 관리 기준

`-e` 옵션으로 환경변수를 직접 전달하는 것과 `--env-file`을 사용하는 것의 차이점 및 실무 관리 방식은 무엇인가요?

핵심 기술 면접 질문

`:latest` 태그 의존의 위험성과 태그 전략

도커 이미지에서 `:latest` 태그만 사용하는 것이 프로덕션 환경에서 위험한 이유와 권장 태그 전략은 무엇인가요?

Dockerfile 분리 vs 멀티 스테이지 타깃 분기

개발 환경(`dev`)과 운영 환경(`prod`)의 컨테이너를 분기할 때 Dockerfile 파일 분리(`-f`)와 멀티 스테이지 타깃 분기(`--target`)의 차이는 무엇인가요?

모범 답변

호스트 아티팩트 복사 vs 멀티 스테이지 빌드

호스트에서 JAR를 빌드해 복사하는 방식은 호스트 머신에 특정 버전의 JDK와 빌드 도구가 설치되어 있어야 하므로 개발자 환경에 따라 빌드 결과가 갈립니다. 반면 멀티 스테이지 빌드는 도커 공식 빌더 이미지 내부에서 컴파일하므로 호스트 환경에 완전히 독립적이며, 최종 산출물에는 경량 JRE만 남아 용량과 보안을 함께 확보할 수 있습니다.

환경변수 주입의 아키텍처 이점 (12-Factor App)

코드와 설정의 분리를 실현합니다. DB 접속 URL이나 비밀번호를 빌드 시점에 고정하면 환경마다 이미지를 새로 빌드해야 하고 보안 자격증명이 노출됩니다. 환경변수로 주입하면 동일한 불변 이미지(Immutable Image)를 외부 클라우드 DBaaS든 로컬 도커 DB든 자유롭게 연결하여 환경 일관성을 달성할 수 있습니다.

명명된 볼륨(Named Volume) vs 바인드 마운트(Bind Mount)

Named Volume은 호스트 경로를 지정하지 않고 도커 엔진이 관리하는 고유 스토리지에 데이터를 격리 보관하므로 이식성과 백업 관리가 뛰어나 데이터베이스(MySQL) 영속화의 실무 표준입니다. 반면 Bind Mount는 호스트 절대경로를 직접 매핑하여 파일을 직접 열람·수정할 수 있어 설정 파일 주입이나 개발 단계 동기화에 적합합니다.

모범 답변

DB 컨테이너 볼륨 미지정 시의 위험성

도커 컨테이너 계층은 기본적으로 휘발성(Stateless)입니다. 볼륨 없이 구동된 MySQL 컨테이너를 삭제(`docker rm`)하는 순간 그동안 쌓였던 모든 테이블과 레코드가 영구 소멸합니다. 따라서 상태를 가지는(Stateful) 데이터베이스는 반드시 Named Volume이나 바인드 마운트를 통해 호스트 스토리지에 데이터를 영속화해야 합니다.

사용자 정의 네트워크 연동의 보안 이점

데이터베이스 포트(3306)를 호스트 외부로 포워딩하면 외부에 DB가 노출될 위험이 있습니다. 사용자 정의 네트워크를 사용하면 외부 포트 개방 없이 동일 가상 사설망 안에서만 컨테이너 이름(`mysql-db`)으로 통신할 수 있어 보안성이 크게 향상되며, IP 변경에 영향받지 않는 내장 DNS 서비스 디스커버리가 지원됩니다.

기본 브리지 vs 사용자 정의 네트워크의 DNS 해석

도커 기본 `bridge` 네트워크는 컨테이너 간 DNS 이름 해석을 지원하지 않아 사실 IP를 직접 지정해야 하지만, 컨테이너 재시작 시 IP가 변경될 수 있어 장애로 이어집니다. 반면 사용자 정의 네트워크에서는 도커 내장 DNS 서버(`127.0.0.11`)가 컨테이너 이름(`--name`)을 동적 사설 IP로 실시간 해석(서비스 디스커버리)해 줍니다.

모범 답변

iptables NAT 기반 포트 포워딩 원리

도커는 호스트 커널의 `iptables` (`netfilter`) 테이블을 제어합니다. `-p 8080:8080` 적용 시 `PREROUTING` 체인에 DNAT(Destination NAT) 규칙을 등록하여, 호스트의 8080 포트로 유입되는 TCP 패킷의 목적지 IP를 컨테이너 고유 가상 사설 IP(예: `172.18.0.x:8080`)로 변환해 브리지 인터페이스로 라우팅합니다.

`-e` vs `--env-file` 관리 기준

`-e`는 터미널 명령줄에 직접 키-값을 작성하므로 단일 변수 주입에는 편리하지만, 변수가 늘어나면 관리가 어렵고 비밀번호가 쉘 히스토리에 노출됩니다. `--env-file`은 설정을 파일로 격리하여 한 번에 로드하므로 명령어가 간결해집니다. 실무에서는 실제 `.env` 파일을 `.gitignore`에 등록하고 템플릿용 `.env.example`만 저장소에 공유합니다.

모범 답변

`:latest` 태그 의존의 위험성과 태그 전략

`:latest`는 고정된 버전이 아니라 언제든지 덮어씌워질 수 있는 가변 포인터입니다. 운영 환경에서 사용할 경우 어떤 변경점이 배포되었는지 추적할 수 없고 롤백 시 이전 정상 상태로 돌아가기 어렵습니다. 따라서 실무에서는 불변(Immutable) 식별자인 시맨틱 버전이나 Git 커밋 해시를 고유 태그로 부여하고, `:latest`는 보조 별칭으로만 복수 지정(`-t`)합니다.

Dockerfile 분리 vs 멀티 스테이지 타겟 분기

환경별로 Dockerfile을 완전히 분리하는 방식(`-f`)은 직관적이지만 기본 패키지나 의존성 설정이 중복되어 관리 부담이 커집니다. 반면 단일 `Dockerfile` 내에서 멀티 스테이지로 타겟(`AS dev`, `AS prod`)을 정의하고 `--target`으로 분기하면 베이스 환경을 공유하면서 필요한 단계만 선택 빌드할 수 있어 유지보수성이 뛰어납니다.

깊이 알아보기

Dockerfile 공식 사양 Dockerfile reference

지시어 전체 목록과 동작 규칙 원문

<https://docs.docker.com/reference/dockerfile/>

멀티 스테이지 빌드 Multi-stage builds

스테이지 분리와 산출물 복사 공식 가이드

<https://docs.docker.com/build/building/multi-stage/>

도커 볼륨 Volumes

명명된 볼륨 생성·마운트·관리 공식 문서

<https://docs.docker.com/engine/storage/volumes/>

바인드 마운트 Bind mounts

호스트 경로 직접 마운트의 특성과 주의점

<https://docs.docker.com/engine/storage/bind-mounts/>

깊이 알아보기

사용자 정의 브리지 Bridge network driver

컨테이너 이름 기반 DNS 해석 공식 설명

<https://docs.docker.com/engine/network/drivers/bridge/>

설정 분리 원칙 12-Factor Config

환경변수로 설정을 분리하는 근거 원문

<https://12factor.net/config>



- EOF -